Triton 优化 Pass Lab

从一个 Triton kernel 出发，沿着 TTIR、TTGIR、LLVM IR、PTX 观察编译优化。

主线例子：blocked GEMM

对照例子：fused softmax

课程重点：data layout、software pipelining、tensor core lowering

Lab 产物

- `kernels.py`：GEMM 和 softmax kernel
- `run_lab.py`：运行 kernel 并导出 IR
- `artifacts/`：TTIR、TTGIR、LLVM IR/LLIR、PTX
- `README.md`：运行步骤和 IR 阅读路线
- `slides.typ`：课堂讲解材料

推荐命令：

```
python labs/triton-opt/run_lab.py --kernel all
```

一张图看编译链

```
Python Triton kernel
  -> TTIR
  -> TTIR canonicalization / CSE / LICM
  -> TTGIR with GPU layout encodings
  -> layout propagation / dot operand optimization
  -> software pipelining / prefetch
  -> LLVM IR or LLIR
  -> PTX with mma.sync or wgmma
```

核心问题：每一层 IR 新增了什么信息，又丢掉了什么高层语义？

为什么用 GEMM 做主线

GEMM 同时覆盖 Triton 编译器最重要的几个优化主题：

- K 维循环适合讲 software pipelining
- `tl.dot` 是 tensor core lowering 的高层锚点
- A/B/C tile 的 layout 选择直接影响内存和计算效率
- shared memory swizzle、bank conflict、MMA operand layout 都能在 IR 中找到线索

GEMM Kernel 结构

```
pid = tl.program_id(axis=0)
offs_m = ...
offs_n = ...
acc = tl.zeros((BLOCK_M, BLOCK_N), tl.float32)

for k_start in range(0, K, BLOCK_K):
    a = tl.load(...)
    b = tl.load(...)
    acc += tl.dot(a, b)

tl.store(..., acc)
```

这段代码天然对应：program mapping、masked load、dot、loop-carried accumulator、store。

为什么加 Softmax 对照

Softmax 不强调 tensor core，但很适合观察 TTIR 层：

- 一行一个 program
- mask 处理非 2 的幂列数
- `load -> max -> exp -> sum -> div -> store` 在单 kernel 中融合
- reduce 和 elementwise 混合，IR 比 GEMM 更短

它帮助学生理解：Triton 优化不只服务 matmul。

TTIR：接近 Triton 语义

在 `00_ttir.mlir` 中找：

- `tt.program_id`
- `tt.load` 和 `tt.store`
- `tt.dot`
- `tt.exp`、`tt.reduce` 或 reduce region
- mask 和 pointer arithmetic
- `scf.for` 或展开后的 K 维循环

TTIR 仍然能看出“程序员写了什么”。

TTIR 层优化

TTIR 主要承接通用编译优化：

- canonicalization：把 IR 规整成更容易匹配的形式
- CSE：合并重复表达式
- LICM：外提循环不变量
- DCE：删除无用计算
- inlining / symbol cleanup：清理 JIT 生成的函数边界

课堂观察点：最终 TTIR 是优化后的快照，不一定展示每个 pass 的前后对比。

TTGIR：加入 GPU Layout

TTGIR 的关键变化：layout encoding 成为 tensor type 的一部分。

常见线索：

- `#ttg.blocked`
- `#ttg.dot_op`
- `#ttg.nvidia_mma`
- shared memory layout encoding
- `ttg.convert_layout`

TTGIR 不只问“tensor 多大”，还问“元素分布在哪些 CTA、warp、thread、register 或 shared memory 上”。

Data Layout 的直觉

同一个 tile 可以有很多物理摆放方式：

```
logical tile:
  [M x N]


physical questions:
  which CTA owns it?
  which warp owns each slice?
  which lane owns each element?
  is it in registers or shared memory?
  does shared memory need swizzle?
```

layout 是 Triton 从高层 tensor 程序走向 GPU 硬件的核心桥梁。

为什么 convert_layout 贵

`ttg.convert_layout` 不是类型标注变化那么简单。

它可能意味着：

- warp shuffle
- shared memory 中转
- register 重排
- 额外 barrier 或 wait
- 更长 live range 和更高寄存器压力

因此 TTGIR 会做 layout propagation、remove conversions、rematerialization。

Layout Propagation

简化模型：

```
anchor ops choose preferred layout
  load/store prefer memory-friendly blocked layout
  dot prefers MMA-friendly operand layout

propagate layout through cheap ops
resolve conflicts
rewrite IR
remove redundant conversions
```

课堂问题：如果 load 和 dot 想要不同 layout，编译器应该在哪个位置转换？

Dot Operand Optimization

GEMM 中最重要的锚点是 `tl.dot`。

TTGIR 会围绕 dot 做：

- 把输入转换为 dot operand encoding
- 为 tensor core 选择 MMA accumulator layout
- 将部分 operand 放入 shared memory
- 使用 swizzle 减少 bank conflict
- 在 Hopper 路径中把 transpose 融入 memory descriptor

Shared Memory Swizzle

为什么 shared memory 需要 swizzle?

```
bad layout:
  many lanes hit the same bank
  -> bank conflict
  -> serialized access

swizzled layout:
  lanes spread across banks
  -> higher bandwidth
  -> tensor core operands arrive faster
```

IR 观察点：shared memory allocation、local load、dot operand layout。

Software Pipelining

目标：让内存访问和计算重叠。

```
no pipeline:
  load k0 -> dot k0 -> load k1 -> dot k1

pipeline:
  preload k0, k1
  dot k0 while loading k2
  dot k1 while loading k3
```

在脚本中调 `--num-stages` 可以改变 pipeline 深度。

Pipeline 的三段式

software pipelining 通常生成：

- prologue：主循环前预填若干 stage
- kernel：稳定状态，多个 iteration 的操作交错执行
- epilogue：主循环后收尾剩余 stage

IR 观察点：

- 新的 loop arguments
- cross-stage values
- circular buffer index
- async copy / wait / barrier

num_stages 的权衡

更大的 `num_stages` 可能隐藏更多延迟，但不是越大越好。

代价：

- shared memory 占用增加
- register live range 变长
- occupancy 可能下降
- barrier 和调度复杂度增加

实验问题：`--num-stages 1/3/5` 下，IR 和性能变化是否一致？

Tensor Core Lowering

从高层到硬件：

```
tl.dot
  -> tt.dot
  -> TTGIR dot with MMA layouts
  -> NVIDIA MMA lowering
  -> LLVM / NVVM
  -> PTX instruction
```

Ampere / Ada：常见 `mma.sync`

Hopper：可能看到 `wgmma.mma_async` 和 wait/barrier

在 PTX 中看什么

搜索关键词：

- `mma.sync`
- `wgmma`
- `ldmatrix`
- `barrier`
- `wait`
- `.shared`

把它们反向映射到高层概念：

`tl.dot`、shared memory tile、pipeline stage、async wait、layout conversion。

学生实验任务

建议四组对比：

- 改 `BLOCK_M/N/K`：tile shape 和 layout 如何变化？
- 改 `num_warps`：warp 分布如何变化？
- 改 `num_stages`：pipeline 结构如何变化？
- 比较 matmul 与 softmax：优化重点为什么不同？

要求学生给出 IR 截图或片段，并解释对应的 pass 作用。

总结

Triton 的优化可以分层理解：

- TTIR：高层 tensor 程序和通用优化
- TTGIR：GPU layout、dot operand、shared memory、pipeline
- LLVM/PTX：硬件指令、同步、寄存器和内存细节

最重要的观念：Triton 性能来自编程模型、layout 系统和后端硬件 lowering 的组合。